

Amortized Bayesian parameter recovery for dynamical systems

with conditional flow matching

The `amortix` package: architecture, evaluation methodology, and benchmarks

Ivan Oseledets

Measurements in this report correspond to package commit `fd36e4d`. The package’s default observation embedding for variable-design problems was changed afterwards (commit `d35f9c4`): the consecutive-pair embedding used here for Markov-observed series is now an explicit option (`embed="wfilm"`), and the default is the bare-point embedding.

Abstract

A package for amortized Bayesian estimation of the parameters of dynamical systems (ODEs, SDEs, and discretized PDEs) from noisy observations at a finite set of time points is presented. A conditional flow-matching model with a learnable input representation is trained once per system on simulated pairs and then returns posterior samples for any observation set, at any number and placement of points, in about 70 milliseconds. On a benchmark suite of fourteen systems, each equipped with an exact or validated reference posterior, twelve are matched to median Fréchet distances of 0.003–0.08 in reference-normalized coordinates. The machinery that builds and validates the reference posteriors is part of the package.

1 Introduction

Recovering the parameters of a differential-equation model from noisy measurements of its solution is one of the basic inverse problems of applied science: a modeler writes down a dynamical system (an ODE, an SDE, or a PDE) whose right-hand side depends on unknown physical constants, observes a trajectory at a finite set of time points, and asks what the observations say about the constants. Like most inverse problems, this one is ill-posed in the practical sense: different parameter values can produce nearly indistinguishable observations, especially when the state is partially observed or the sampling is sparse, so a point estimate both hides the ambiguity and is unstable under noise. The Bayesian formulation resolves this by reporting the full posterior distribution over parameters given the data, so that ambiguity becomes posterior spread and correlation, at the price of two computational obstacles. First, for most stochastic and partially observed systems, the likelihood of the observations requires integrating over unobserved trajectory segments and is intractable or expensive. Second, modern workflows need posteriors *repeatedly*: calibration studies process thousands of synthetic observation sets, sequential experiments update the posterior after every new measurement, and Bayesian experimental design compares candidate observation schedules by the posteriors they would produce. Per-instance MCMC or optimization inside such loops is the dominant cost even when a likelihood is available.

Simulation-based inference (SBI) addresses both issues by learning a conditional generative model of the parameters given the data from simulated pairs (parameters drawn from the prior, observation sets produced by the simulator), thereby amortizing the cost of inference into a one-time training phase (Cranmer et al., 2020). Neural posterior estimation (NPE) and its sequential variants (Papamakarios and Murray, 2016; Greenberg et al., 2019) fit normalizing flows by maximum likelihood; flow-matching and diffusion variants (Wildberger et al., 2023; Gloeckler et al., 2024) replace the density estimator with transport-based generative models. Mature software exists: the `sbi` toolkit (Boelts et al., 2024), BayesFlow (Radev et al., 2020), and the `sbim` benchmark collection (Lueckmann et al., 2021).

These toolkits leave three things unsolved in the dynamical-systems setting, and the three are connected. First, the conditional density estimators consume a fixed-length data vector after global standardization: the estimator sees whatever coordinates the user chose. On multiplicative and heavy-tailed processes (prices, concentrations, populations) we found this convention to fail in reproducible ways: with prescribed coordinates the estimator ends up biased or mis-calibrated (Sec. 3.3). Second, a fixed-length interface cannot produce the object that sequential refinement and Bayesian experimental design consume: one

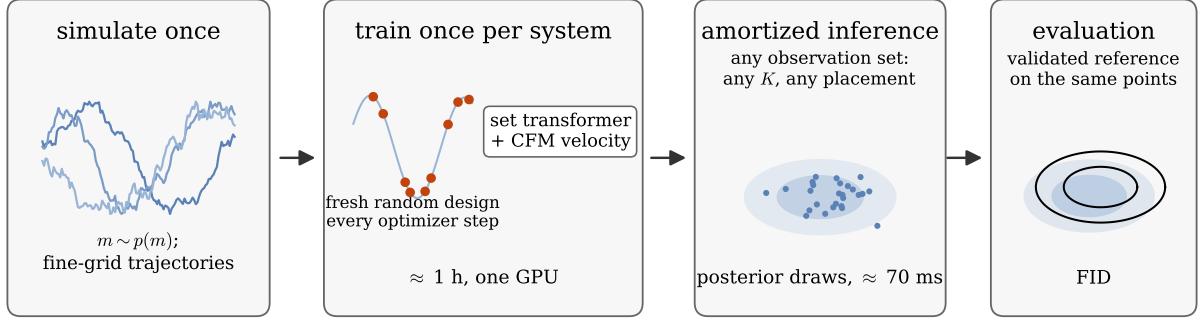


Figure 1: The approach. Parameters drawn from the prior are simulated once on a fine grid (left). Training fits a conditional flow-matching posterior with a set-transformer encoder, drawing a fresh random observation design at every optimizer step (Secs. 3, 4). The trained network returns posterior samples for any observation set of the experiment, at any number and placement of points. Accuracy is measured against exact or validated reference posteriors computed on the same observed points (Sec. 5).

posterior conditioned on an arbitrary subset of the measurement points. Third, the standard evaluation practice, simulation-based calibration (Talts et al., 2018), is a necessary-condition test whose statistical power at practical sample sizes is limited, and learned posteriors can be confidently wrong in ways that rank diagnostics do not detect (Hermans et al., 2022), so a failure of the first kind can pass the accepted diagnostics.

This report describes **amortix**, a compact PyTorch package (about 3,500 lines; one default model of 4.8×10^5 weights) built around three decisions that answer the three gaps in order: the input representation is learned as part of the model, one network is trained over random observation designs, and accuracy is measured against reference posteriors rather than self-consistency. Our main contributions are:

- *A learnable input representation* (Sec. 3): a monotone mixture-of-scales reparametrization of the value axis, attention phases computed from the physical observation times, and explicit design-size features. Each component is motivated by a measured failure in the conditioning of the encoder input and carries a learnable repair, documented with the component itself (Sec. 3.3).
- *Design amortization* (Secs. 4, 7.3): one network approximates $p(m | S)$ for every subset S of the measurement points (arbitrary sizes and placements, including spatio-temporal sensor designs for PDEs), together with the training procedure this requires: designs resampled fresh at every optimizer step, under a measured design-size law.
- *An evaluation methodology with references as the record* (Sec. 5): exact or validated reference posteriors for all fourteen benchmark systems, built by the engine the likelihood admits (closed forms, adaptive Metropolis, nested sampling, tempered SMC over particle-filter estimates), admitted by three independent checks, with simulation-based calibration used only as a screen; the machinery ships with the package.
- *Measurements* (Sec. 7): the accuracy record on the suite and its dependence on model size, simulation budget, and data reuse; and the full cost accounting (Table 3): training in 58–82 minutes per system on one GPU, bit-identical on a laptop CPU at four times the wall time; a posterior in ~ 70 ms thereafter, against milliseconds to an hour per observation set for the per-instance reference samplers. For a single dataset with a cheap likelihood, classical sampling remains the right tool; the case for amortization is repetition and intractable likelihoods.

Appendix D maps the terminology of this report to package options.

2 Method

Figure 1 shows the approach as a whole; this section defines each element.

2.1 Problem formulation

Let a dynamical system depend on a parameter vector $m \in \mathbb{R}^d$ with a known prior $p(m)$, uniform on a box in all benchmarks below. The system produces a state trajectory $x(t)$, through an ODE $\dot{x} = f(x; m)$, an SDE $dx = f(x; m)dt + \sigma(x; m)dW_t$, or a spatially discretized PDE, and the trajectory is observed through a noisy channel at K points:

$$y_i = h(x(t_i)) + \varepsilon_i, \quad \mathcal{D} = \{(t_i, y_i)\}_{i=1}^K. \quad (1)$$

We call $\mathcal{D}$ the *observation set* of one experiment; it is the entire input to inference. Training and evaluation below involve many independently simulated observation sets, each paired with the parameter that generated it. The observation operator h may expose only part of the state: in the benchmarks below, only the price of an asset while its stochastic volatility is latent, only the membrane voltage of a neuron while its gating variables are latent, or only three spatial sensors of a PDE field. The target is the posterior $p(m | \mathcal{D}) \propto p(\mathcal{D} | m)p(m)$.

Three standing assumptions delimit the setting. The parameter is finite- and low-dimensional ($d = 2-5$ in every benchmark below) with a prior that factorizes across components and has tractable marginal CDFs (uniform boxes throughout; the construction extends to any such prior). The simulator can be run forward cheaply and repeatedly, but nothing else is required of it: no likelihood evaluations, no gradients. Observation times (and sensor indices) are known exactly; the measurement noise ε_i is independent across observations with a known per-system law.

Although the posterior itself is intractable, sampling from the *joint* distribution of parameters and data is easy: draw $m_j \sim p(m)$, simulate the system, apply the observation channel. This yields arbitrarily many training pairs $(m_j, \mathcal{D}_j)$, and a conditional generative model trained on them approximates the posterior for every observation set simultaneously: the cost of inference is moved into a one-time training phase, and inference for a new observation set is a forward pass. We impose one requirement beyond standard amortization: a single trained network must remain correct for any number K and any placement of observation times (and, for PDEs, sensor positions). Formally, for every subset S of the measurement points of one experiment the network must approximate $p(m | S)$, and the family of answers must be coherent; in particular the posterior must widen as points are removed. We refer to this requirement as *design amortization*; it is the object consumed by sequential refinement and by Bayesian experimental design, where candidate observation schedules are ranked by the posteriors they would produce.

2.2 Posterior sampling by conditional flow matching

The idea of the method is to represent the posterior not by a density formula but by a *transport*: a learned, data-conditioned deformation that carries a fixed reference distribution onto the posterior. Sampling then amounts to drawing from the reference and applying the deformation: there is no normalizing constant to compute and no invertibility constraint to build into the network, because the deformation is defined implicitly as the flow of a velocity field, and the field itself is fitted by ordinary least-squares regression on simulated pairs. This construction is conditional flow matching (Lipman et al., 2023).

Concretely, parameters are first reparametrized componentwise,

$$m \mapsto \Phi^{-1}(F_{\text{prior}}(m)) \in \mathbb{R}^d, \quad (2)$$

where F_{prior} collects the marginal prior CDFs and Φ is the standard normal CDF. In the new coordinates the prior is exactly $\mathcal{N}(0, I)$ and box constraints disappear; usefully, the uninformative limit then costs nothing: when the data carry no information the posterior coincides with the reference and the transport is the identity. From here on we work exclusively in these coordinates and keep writing m for the parameter in them; the inverse of (2) is applied exactly once, at the end of sampling, to return draws on the original scale.

The transport is indexed by a *virtual time* $\tau \in [0, 1]$, an algorithmic variable with no relation to the physical time t at which the dynamical system is observed; t appears only inside observation sets, τ only in the transport. Subscripts in τ denote the position along the transport path: m_1 is a parameter drawn jointly with its observation set $\mathcal{D}$ (the endpoint), $m_0 \sim \mathcal{N}(0, I)$ is pure reference noise (not the transform of any parameter, but the raw material that the transport turns into a posterior draw by $\tau = 1$), and between them lies the training interpolation $m_\tau = (1 - \tau)m_0 + \tau m_1$. The velocity network is trained to predict the direction of this path by regression:

$$\mathcal{L}(\theta) = \mathbb{E}_{m, \mathcal{D}, m_0 \sim \mathcal{N}(0, I), \tau \sim U[0, 1]} \|v_\theta(m_\tau, \tau, \mathcal{D}) - (m_1 - m_0)\|^2. \quad (3)$$

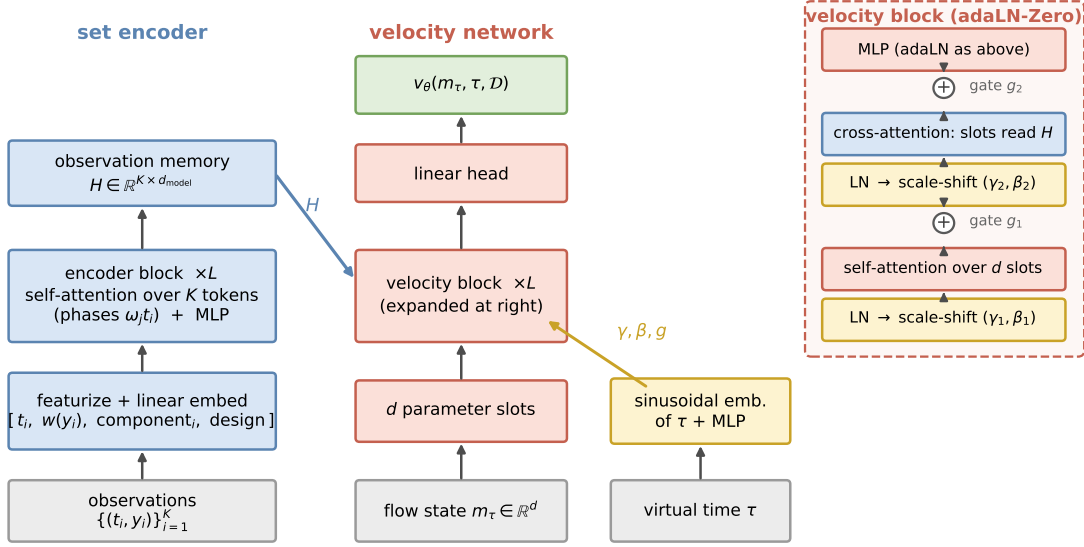


Figure 2: Model architecture; arrows denote data flow, bottom to top. Left: each observation is featurized and linearly embedded into one token; L transformer encoder blocks, whose attention phases are computed from the physical observation times (Sec. 3.2), produce a memory H of K encoded states. Middle: the d components of the flow state m_τ form d parameter-slot tokens processed by L velocity blocks and a linear head into the velocity v_θ . Right: one velocity block expanded — self-attention across parameter slots, cross-attention reading the observation memory H , and an MLP, each preceded by layer normalization with scale-shift (γ, β) and followed by a gated residual connection (g , marked $+$); the triples (γ, β, g) are produced from the virtual time τ by a zero-initialized map, so the untrained block is the identity (Peebles and Xie, 2023).

The minimizer of (3) is the marginal velocity field whose flow transports $\mathcal{N}(0, I)$ at $\tau = 0$ to $p(m_1 | \mathcal{D})$ at $\tau = 1$ (Lipman et al., 2023). Sampling draws $m_0 \sim \mathcal{N}(0, I)$, integrates $dm/d\tau = v_\theta(m, \tau, \mathcal{D})$ from $\tau = 0$ to 1, and maps the endpoint back through the inverse of (2); the numerical settings of the integration are implementation details, collected in Sec. 4. The same estimator family, with a different conditioning architecture, is available in **sbi** as flow-matching posterior estimation (FMPE) (Wildberger et al., 2023); this package differs in the input representation rather than in the generative engine.

No likelihood evaluations, simulator gradients, or per-instance optimization are required at any stage; the simulator is executed only forward, during training-set generation.

The training objective is consistent in the following sense.

Proposition 1 (consistency). *Let $(m, \mathcal{D}) \sim p(m)p(\mathcal{D} | m)$ and let m_1 be given by (2). If the loss (3) is minimized over all measurable velocity fields, then for $p(\mathcal{D})$ -almost every $\mathcal{D}$ the flow of the minimizer transports $\mathcal{N}(0, I)$ at $t = 0$ to the conditional law $p(m_1 | \mathcal{D})$ at $t = 1$; equivalently, the sampling procedure above draws exactly from the posterior (Lipman et al., 2023; Wildberger et al., 2023).*

The proposition is an idealization: with a finite network, a finite simulation budget, and a numerical solver, no calibration guarantee survives, and the residual error becomes an empirical quantity. Section 5 builds the instruments that measure it.

3 Conditioning architecture

The velocity field $v_\theta(m, \tau, \mathcal{D})$ must be conditioned on an observation set of time-stamped measurements, and three properties of that set drive the architecture: the number of observations K varies from experiment to experiment (by two orders of magnitude in the variable-design setting); the observation times are arbitrary real numbers, in general not equally spaced, so positional schemes built for uniform sequences do not apply; and the times carry all ordering information, so the output must not depend on how the observations happen to be stored. These properties rule out the fixed-length-vector conditioners that are the default in NPE implementations, and they are naturally met by a transformer operating on one feature vector per observation, called a *token* throughout this section.

3.1 Overall scheme

The network evaluates $v_\theta(m, \tau, \mathcal{D})$, where m is the normalized parameter vector partway along its transport path (the interpolation m_τ at training time, the current ODE state at sampling time) and τ is the virtual time of the transport. The design follows the encoder–decoder pattern that is standard in conditional flow-matching and diffusion models: the data are encoded once, and a decoder combines that encoding with the object being generated,

$$v_\theta(m, \tau, \mathcal{D}) = \text{Dec}(m, \tau, \text{Enc}(\mathcal{D})), \quad (4)$$

with explicit types: Enc maps the observation set, the K time-stamped measurements of one experiment, to a memory of K vectors, $\text{Enc}(\mathcal{D}) \in \mathbb{R}^{K \times d_{\text{model}}}$, computed once per observation set, and $\text{Dec} : \mathbb{R}^d \times [0, 1] \times \mathbb{R}^{K \times d_{\text{model}}} \rightarrow \mathbb{R}^d$ combines the current transport point and the virtual time with that memory. Two differences from the image and text instances of this pattern dictate the concrete form (Fig. 2). First, the conditioning is not a fixed-size embedding but an observation set whose size K varies: Enc is therefore a transformer over per-observation tokens, its output is a *memory* of K encoded states rather than one pooled vector, and Dec reads the memory by cross-attention, the one standard mechanism indifferent to K . Second, the generated object is not an image but a short vector of d parameters, so the decoder is small: the d components of m are treated as d tokens, self-attention among them expresses posterior correlations (the ridge of the Fisher–KPP posterior, the drift–volatility coupling of GBM), and the virtual time τ enters every block through zero-initialized adaptive layer normalization (adaLN-Zero), the conditioning mechanism of diffusion transformers (Peebles and Xie, 2023), so that the untrained network is an identity-like map.

On the encoder side, observation i is described by a few features and lifted to the model width by a learned linear map. The features are its time t_i ; its transformed value $w(y_i)$, with w the learnable monotone map of Sec. 3.3; an index identifying the sensor or state component that produced the value (multivariate states contribute one token per observed component per time); and the design-size feature of Sec. 3.3. The attention mechanism that lets these tokens interact is the subject of the next subsection; the choice of features is the subject of Sec. 3.3.

This scheme is the survivor of a sequence of measured alternatives; the failed ones are retained in the package as reproducible controls. All sizes are configuration parameters. One default configuration (width 64, four heads, three blocks per side) is used unchanged for every experiment in this report; at the training-set sizes we use, models of this scale proved sufficient (doubling width and depth changes no benchmark result), and nothing in the construction caps the scale.

3.2 Attention phases from observation times

We adapt rotary position embeddings, the standard positional scheme of modern transformers (Su et al., 2024), to irregular time sampling.

First the classical construction. For token states $x_1, \dots, x_K \in \mathbb{R}^{d_{\text{model}}}$ entering an attention layer, each head computes

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad u_i = W_V x_i, \quad W_Q, W_K, W_V \in \mathbb{R}^{d_h \times d_{\text{model}}}, \quad (5)$$

and token i aggregates the others with weights $a_{il} = \text{softmax}_l(\hat{q}_i^\top \hat{k}_l / \sqrt{d_h})$, where $\hat{q}, \hat{k}$ are position-modulated: each query and key is split into $d_h/2$ planar blocks $q_i^{(j)} \in \mathbb{R}^2$ and block j is rotated by an angle set by a per-token scalar ϕ_i ,

$$\hat{q}_i^{(j)} = R(\omega_j \phi_i) q_i^{(j)}, \quad \hat{k}_i^{(j)} = R(\omega_j \phi_i) k_i^{(j)}, \quad R(\phi) = \begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix}. \quad (6)$$

In the classical scheme the scalar is the token’s integer position in the sequence, $\phi_i = n_i$, and the frequencies form a geometric progression across blocks, $\omega_j = \omega_{\text{max}}(\omega_{\text{min}}/\omega_{\text{max}})^{(j-1)/(d_h/2-1)}$, so that high-frequency blocks resolve neighboring positions and low-frequency blocks long ranges. Because rotations preserve inner products up to the relative angle,

$$\hat{q}_i^\top \hat{k}_l = \sum_{j=1}^{d_h/2} (q_i^{(j)})^\top R(\omega_j(\phi_l - \phi_i)) k_l^{(j)}, \quad (7)$$

every attention logit depends on the positions only through differences $\phi_l - \phi_i$: relative position awareness with no positional parameters.

For irregularly sampled data the integer position is meaningless, and the generalization is immediate: we set $\phi_i = t_i$, the physical time stamp of observation i , and choose the frequency range so that the periods $2\pi/\omega_j$ span from the finest observable time gap to several times the horizon. Nothing else changes; by (7), every logit now depends on the time stamps only through differences $t_l - t_i$. The substitution of physical time for position is the same one that time-aware attention models for irregular clinical and event series make in other positional schemes (Horn et al., 2020; Shukla and Marlin, 2021). Two consequences follow at once: the encoder is exactly permutation-invariant (relabeling tokens leaves the right-hand side of (7) unchanged; verified numerically to 10^{-8}), and variable designs require no positional bookkeeping. The phases enter only the encoder’s self-attention; the cross-attention from parameter slots to the observation memory is unmodified attention, since parameter slots carry no time stamp and the memory states already encode temporal structure.

Integer positions, by contrast, create an implicit train/test contract on token placement: we found that a model trained on spread-out subsampled positions and evaluated on contiguously packed ones acquires substantial posterior biases while every component remains individually correct. With physical-time phases the contract does not arise.

3.3 Input features

The remaining design choices concern what the tokens contain. Three of them matter and are described in turn: a learnable transform of the observed values, an optional increment-based token construction for directly observed diffusions, and an explicit design-size feature. Each occupies a few lines of code; each repairs a measured failure in the conditioning of the numbers presented to the set encoder, and each repair is a learnable module rather than a problem-specific transformation. The same

Value transform. State variables of the benchmark systems live on very different scales, and some (prices, concentrations, populations) are multiplicative: the information sits in relative rather than absolute changes, and the coordinate in which such a variable is well-behaved is logarithmic. We found that ignoring this is costly: on GBM, raw price tokens under global standardization force the network to divide squared increments by squared levels internally, and the result is a persistent volatility bias that no amount of training removes. Rather than prescribing coordinates per problem, each value enters its token through a learned monotone map,

$$w(x) = \sum_{k=1}^S a_k \operatorname{sign}(x) \log(1 + |x|/s_k), \quad a_k \geq 0. \quad (8)$$

Each basis function is linear for $|x| \ll s_k$ and logarithmic for $|x| \gg s_k$, so the family contains the identity, the signed logarithm, and every compression in between; positivity of the weights makes w monotone by construction; and, most importantly, every candidate compression is present from initialization, so training only reweights fixed nonlinearities, and the downstream network can exploit whichever coordinate helps from the first optimizer step. The scales s_k sit on a fixed geometric grid covering the dynamic range of the data ($S = 8$ octaves by default, one setting for all fourteen systems, no per-problem adjustment).

This structure was compared against simpler-looking learnable alternatives on geometric Brownian motion, where the correct coordinate (the log-price) is known, so a failure to find it is visible against the closed-form posterior. A generic two-layer MLP on the raw values and a random-Fourier-feature map both fail to find the coordinate at matched budget: a free-form map must discover the shape and the scale of the compression simultaneously, from gradients arriving through the whole downstream network, and in practice it does not. The Yeo–Johnson power transforms (Yeo and Johnson, 2000), a classical one-parameter family (in essence $x \mapsto ((1 + |x|)^\lambda - 1)/\lambda$ applied with sign symmetry, which passes through the identity at $\lambda = 1$ and tends to the logarithm as $\lambda \rightarrow 0$), also fail, for an instructive reason: trained end-to-end, λ never leaves its initialization, because a single global exponent must traverse a gradient plateau before the downstream network has learned anything that would reward the move. What decides the outcome is therefore the parametrization rather than the hypothesis class: the mixture (8) finds the coordinate reliably, with the trained weights concentrating on the small-scale components, i.e. the model learns the log-price.

Increment features for directly observed diffusions. For scalar SDEs whose state is observed directly, with no latent components, the package offers a second token construction: after sorting by time,

consecutive observations contribute

$$\left[t_{(i)}, w(y_{(i)}), \Delta w_i, (\Delta w_i)^2, w_t(\Delta t_{(i)}) \right], \quad \Delta w_i = w(y_{(i+1)}) - w(y_{(i)}).$$

The motivation is elementary: for a Markov observed process the likelihood factorizes over consecutive pairs, transition densities of diffusions depend on increments and gaps, and squared increments carry the quadratic-variation information that identifies diffusion coefficients. This is an inductive bias of the same character as convolutions for images: useful where its assumption holds, not a requirement of the method; the point-token path of Sec. 3.1 is fully general.

The rule for choosing between the constructions (increment tokens exactly when the observed process is Markov) was tested in both directions: an increment-token model transfers across Markov systems without adjustment, and on the Heston model, whose observed price is not Markov because the volatility is latent, increment and point tokens perform identically, as the factorization argument predicts. The package selects the construction from a per-problem flag.

Two further normalization details matter for processes with jumps, where squared increments within one observation set span several orders of magnitude. A running feature normalization then takes its scale from the rare outliers and never converges; the representation drifts during training and the flow trains against a moving target, which we found produces a substantial volatility bias on the Merton jump-diffusion. Probing the encoder showed the failure to be one of representation stability rather than extraction: the failing configuration encodes robust volatility statistics as well as the healthy one. Two repairs follow. First, increment features are logarithmically compressed before normalization; this is a near-identity on systems without jumps, since $\log(1+x) \approx x$ for small x , and in our experiments it removes the bias. Second, a smaller residual remains whose mechanism is a jump-classification threshold that floats with the unknown volatility; to remove it, the normalization is made adaptive to the observation set: robust summary statistics s of the set, pooled over non-padding positions, are mapped by a zero-initialized MLP to per-channel scale and shift corrections,

$$h_c \mapsto (1 + \gamma_c(s)) h_c + \beta_c(s) \tag{9}$$

— a feature-wise affine modulation, FiLM in the terminology of Perez et al. (2018). We found that it removes the residual as effectively as a hand-crafted per-observation-set rescaling, and that three conditions are required for it to help: zero initialization, modulation applied *after* the running normalization, and pooled statistics that exclude padding positions.

Design size. Posterior width is governed by the number of observations (in regular models the error scales as $K^{-1/2}$), while softmax attention computes normalized weighted means and is therefore nearly invariant to the size of the set it aggregates. Probing the encoder confirms the mechanism: every variant we tested recovers posterior-mean statistics from the pooled output, while how well the pooled output encodes $\log K$ varies widely across variants and ranks them exactly in the order of their width errors. We found that supplying this one number as an explicit feature, $\log K$ in every token, removes most of the width error at dense designs, and that the design-resampling training procedure of Sec. 4 closes the rest. Appending a few learned register tokens (Darcet et al., 2024) achieves the same effect without an explicit feature.

4 Training procedures

Common settings. Sampling integrates the velocity field with a fixed-step midpoint scheme, 20 steps by default; the step count was fixed once by a convergence check (fourth-order Runge–Kutta with three times as many steps changes no benchmark statistic), and evaluation uses an exponential moving average of the training weights (decay 0.999). The default optimizer is Muon (Jordan et al., 2024): the momentum of every two-dimensional weight is orthogonalized by a Newton–Schulz iteration before it is applied, with Adam retained for biases, normalizations, and embeddings. Its step size follows a measured batch-size rule $\eta(B)$ (Appendix B); the schedule is cosine with linear warmup, and gradients are clipped. At every optimizer step the virtual times are drawn afresh, stratified over $[0, 1]$: $\tau_i = (i - 1 + u_i)/B$ for a batch of size B , with $u_i \sim U[0, 1]$ i.i.d., $i = 1, \dots, B$; fresh reference noise m_0 is drawn with them, so no pairing between observation sets and virtual times is ever repeated. For fixed-design problems the training set consists of n simulated observation sets and training runs for a fixed number of epochs over it; the calibration-screen gallery of Appendix A uses $n = 40,000$ simulations and 35 epochs.

Design amortization. Training data are simulated once on a fine grid; observation designs are then resampled *from the stored fine trajectories at every optimizer step*. The procedure has three components, each fixed by a controlled comparison on GBM, Ornstein–Uhlenbeck, and FitzHugh–Nagumo:

1. *Fresh designs at every step.* Re-drawing each trajectory’s observation subset at each step makes design memorization impossible and lets one simulation serve arbitrarily many designs. The alternative, one fixed design per trajectory, closes the dense-design error at long budgets but overfits the design set: in our experiments it produces overconfident posteriors at sparse K .
2. *The design-size law.* K is drawn 50% log-uniform on $[K_{\min}, K_{\max}]$ and 50% uniform on the dense half of the range. Log-uniform sampling alone puts too little mass at large K and leaves dense designs under-trained; the mixture removes the effect.
3. *Budget scaling.* When higher accuracy is required, simulations and optimizer steps are scaled together at constant epochs; in our experiments the residuals then decrease steadily with this single budget variable, while passes over reused observation vectors eventually destroy accuracy, which is why the scaling must be joint.

With this procedure, we found the posterior width to match the exact per-design posterior closely and uniformly over the whole range of design sizes.

Budgets and stopping. Step budgets are set with deliberate slack, and the shipped weights are chosen by validation: training monitors a validation set (built by the same machinery as the test evaluation sets, from disjoint seeds) and replaces the final weights with the best monitored checkpoint only when the improvement exceeds the monitor’s own resolution and the monitored value itself sits at least three floors above it, since a monitor asked to rank values of its own resolution returns noise in both directions. With that margin in place, overshooting a budget costs time, never quality, so the step budget is not a per-system quantity to tune. The measurements behind the margin, including the two failure modes of the naive best-checkpoint rule, are in Appendix B.

5 Evaluation methodology

Accuracy is measured throughout against reference posteriors computed on the same observed points, so that a number in this report always says how far the network’s posterior is from the posterior the data actually imply, rather than how well it satisfies a self-consistency condition.

5.1 Reference posteriors as the record

Wherever a tractable likelihood exists, we compare distributions on the same observed points. The headline statistic is a single number per observation set, the squared Fréchet distance between Gaussian fits of the two sample sets in reference-normalized coordinates (each parameter divided by the reference posterior standard deviation),

$$d_F^2 = \|\mu_a - \mu_r\|^2 + \text{tr}(\Sigma_a + \Sigma_r - 2(\Sigma_r^{1/2}\Sigma_a\Sigma_r^{1/2})^{1/2}) \quad (10)$$

— the construction used as the FID score in the image-generation literature, and we adopt that name. The quantity is dimensionless only after a choice of scale; two are used. Writing $\sigma_{r,i}$ for the reference posterior’s standard deviation in coordinate i and Δ_i for the width of the prior box,

$$\text{FID}_\sigma = d_F^2(\{m_a/\sigma_r\}, \{m_r/\sigma_r\}), \quad \text{FID}_\Delta = d_F^2(\{m_a/\Delta\}, \{m_r/\Delta\}), \quad (11)$$

that is, every coordinate is divided by the reference posterior’s own standard deviation (FID_σ , the default; an unadorned FID anywhere in this report means FID_σ) or by the prior range (FID_Δ). They answer different questions, error relative to what the data determine versus error relative to the scale of the problem, and a model of fixed absolute accuracy reads very differently in the two wherever the posterior is narrow (Sec. 7.3).

FID is zero for a perfect match, dimensionless, and combines the three ways a posterior can be wrong: location, width, and orientation error. For scale, a bias of 0.1 reference-sd in each of d dimensions gives FID $0.01 d$, and a posterior 30% too wide in one dimension gives $\text{FID} \approx 0.09$. Where the direction of a failure matters (a posterior that is too wide is conservative, while one that is too narrow misleads without


```

# ---- training a design-amortized posterior -----
m_all, X_all = simulate(prior, n)      # full fine-grid trajectories, once
theta_best, F_best = None, inf
for step in range(T):                  # step budget T, set with slack
    m1, X = batch(m_all, X_all, B)     # stored trajectories, reused
    tokens = tokenize(X, fresh_designs(B)) # per trajectory: K from the mix
                                           # law, K random (time, sensor) points
    tau = stratified(B)                 # fresh virtual times, stratified on [0,1]
    m0 = randn(B, d)                   # fresh reference noise
    z = (1 - tau) * m0 + tau * m1
    loss = norm(v(z, tau, Enc(tokens))) - (m1 - m0) ** 2
    muon_step(loss, lr=eta(B))          # batch step rule, cosine schedule
    ema.update()
    if step % Delta == 0:               # score the EMA weights on validation
        F = monitor(ema, val)
        if F < F_best: theta_best, F_best = copy(ema), F
# ship the best checkpoint only when it clears the monitor's own resolution
if F - F_best > phi and F_best >= kappa * phi: return theta_best
return ema

# ---- inference: n posterior draws for one observation set S -----
C = Enc(tokenize(S))                   # encode once; any size, any placement
z = randn(n, d)                        # or the learned base, where enabled
h = 1 / L                              # L midpoint steps on [0,1]
for tau in arange(0, 1, h):            # all n draws batched
    z_mid = z + h/2 * v(z, tau, C)
    z = z + h * v(z_mid, tau + h/2, C)
return denormalize(z)

```

Figure 3: Training (top) and inference (bottom). Package defaults: the step rule $\eta(B)$ of Sec. 4, monitor period $\Delta = T/24$, resolution multiple $\kappa = 3$ with monitor floor φ (Appendix B), and $L = 20$ ODE steps, fixed once by the convergence check of Sec. 4.

warning) we additionally quote the per-parameter decomposition: the signed posterior-mean error in reference-sd units (*bias*) and the ratio of standard deviations (*width*; 1 exact, above one conservative, below one overconfident).

References are built by the engines of the next paragraph, with construction detailed in Appendix C. Every MCMC reference is validated by a second chain from a well-separated start: a reference is used only once its two chains agree, and the between-chain posterior-mean discrepancy, in posterior-sd units, is the floor below which nothing can be measured, quoted alongside the model’s value throughout. That floor scales with chain length: the Merton jump-diffusion needs 2×10^5 -draw chains to clear it (two chains then agreeing to 0.016 sd in the median, 0.060 at worst); a system whose chains did not converge at any affordable length would be reported under the calibration screen instead; after the engine upgrades of the next paragraph, no system of the suite remains in that state.

Reference samplers and their admission. For nonlinear ODEs and SDEs, obtaining trustworthy reference posteriors is itself a nontrivial task; for every system several approaches were tested, and the per-problem methods below were adopted. The choice of sampler follows the structure of the likelihood. Posteriors are drawn exactly where a closed form exists (linear-Gaussian; GBM through its conjugate structure). Where the likelihood is exact but the posterior is not, adaptive Metropolis suffices for the well-conditioned systems (Ornstein–Uhlenbeck, Cox–Ingersoll–Ross through its noncentral χ^2 transitions, SEIR, Merton, pharmacokinetics, Fisher–KPP); FitzHugh–Nagumo and Hodgkin–Huxley have needle-thin or multimodal posteriors: their sharpest evaluation instances concentrate the posterior to 0.4% and 1.0% of the prior range in the narrowest coordinate, and adaptive Metropolis chains from separated starts disagreed on them by hundreds of posterior standard deviations; these two use nested sampling with *dynesty* (Skilling, 2006; Speagle, 2020) instead. On the median instances the posteriors are unremarkable ($3\text{--}30\times$ narrower than the prior); the needle is a property of the sharp tail, which is exactly where chain methods fail. Double well and polynomial drift have no transition density in closed form; their likelihood is an unbiased bridge estimate (Durham and Gallant, 2002) inside pseudo-marginal MH (Andrieu et al.,

```

import torch
from amortix import FlowPosterior, DESIGN_ZOO

prob = DESIGN_ZOO["heston"]()          # or any DesignProblem subclass
post = FlowPosterior(prob)             # embedding/roPE resolve from the class
post.fit(n_train=120_000, steps=72_000, # about 1 h on one GPU
         retokenize=prob.make_retokenizer()) # fresh designs at every step

gen = torch.Generator().manual_seed(0)
m, raw = prob.simulate(1, gen)         # here: one synthetic observation set
tidx, cidx = prob.sample_design(gen, 20) # 20 (time, sensor) points
tokens = prob.tokens_for(raw[0], tidx, cidx, gen)
draws = post.sample(tokens, n=2000)    # [2000, d] draws, milliseconds

```

Figure 4: The interface for the procedure of Fig. 3: a benchmark system from the design zoo, training with the package defaults, and posterior draws for one observation set. The correspondence between the terms of this report and the package options is tabulated in Appendix D.

2010). Stochastic Lotka–Volterra and Heston are the two systems whose likelihood is available only as a Monte-Carlo estimate rather than a computable number, so nested sampling, whose volume shrinkage assumes a deterministic likelihood, does not apply to them; their references come from adaptive tempered SMC (Del Moral et al., 2006) over an unbiased particle estimate, admitted by population doubling. Because the estimate enters a pseudo-marginal kernel, its own variance is a property of the reference rather than a nuisance: the log of an unbiased estimator is biased downward by roughly half its variance, and where that variance varies over the parameter space the bias tilts the posterior. Both systems are therefore run at a particle count high enough that the estimator’s spread is a fraction of a nat, verified by repeating the estimate at fixed parameters.

Every reference then passes three checks before it is used. Two independent runs must agree below 0.25 posterior sd; the likelihood must reproduce the simulator (on noiseless data its value at the generating parameters is zero to machine precision); and the generating parameters must lie inside the posterior. The three are not redundant: each caught a wrong likelihood in this work that the other two passed, and the third is the one that catches a reference describing a different model than the simulator. Across the suite’s frozen evaluation sets the generating parameters sit at a median 0.5–0.8 posterior sd from the reference mean, with 0–1% of coordinates beyond three (Appendix B).

Heston is the sharpest case of the third check, because the failure is invisible to the first two. Its reference was first built from the idealized transition of the continuous-time model, which is a different process from the one that generated the data: the simulator integrates the variance by clamped Euler, so past the Feller condition the variance spends real time pinned at zero, and the discrepancy grows with the vol-of-vol ξ . On instances generated with $\xi \approx 0.9$ the reference placed ξ near 0.27 at half the true width, the two independent chains agreed on that wrong answer to 0.17 sd, and only the generating parameter, sitting 8.1 posterior sd outside its own reference, exposed the defect. Replacing the idealized transition by a bootstrap particle filter that runs the simulator’s own update rule, admitted against two closed forms in the $\xi = 0$ limit, moved the worst deviation to 2.6 sd, tightened the two-chain agreement to 0.06 sd, and halved the set’s floor; measured against the corrected reference, Heston’s dependence on model size is unremarkable (Sec. 7).

Resolution of the estimator. Both sample sets are finite, so (10) is positive even for a perfect posterior, and the size of that offset sets what the metric can resolve. We measured it directly: drawing *two independent sample sets from the same exact GBM posterior* and scoring one against the other gives a median FID of 0.0157 at 500 samples, 0.0038 at 2,000, 0.0009 at 8,000, and 0.0002 at 32,000: a clean $1/n$, as the estimation error of the fitted means and covariances predicts. Two consequences follow. Differences below ≈ 0.005 at $n = 2,000$ (≈ 0.002 at the 4,000 used to score the evaluation sets of Table 2) are not resolved, so values in that range are reported as “at the resolution limit” rather than compared. And the best values in this report sit above it rather than at it: the design-amortized GBM model reads 0.0107 at $n = 2,000$, 0.0100 at 8,000, and 0.0072 at 32,000 on the same evaluation set, so after subtracting the estimator offset its excess over the reference is ≈ 0.007 at every sample size, a real error of the model rather than an artifact of the finite sample count.

Equation (10) is the squared 2-Wasserstein distance between Gaussian fits; the division by the

system	type	d	obs.	design	reference posterior
linear-Gaussian	linear map	4	full	variable	exact draws, no MCMC
geometric Brownian motion	SDE	2	full	variable	conjugate closed form
Ornstein–Uhlenbeck	SDE	2	full	variable	exact-likelihood MCMC
Cox–Ingersoll–Ross	SDE	3	full	variable	exact-likelihood MCMC
double well	SDE	3	full	variable	bridge, pseudo-marginal MH
polynomial-drift SDE	SDE	5	full	variable	bridge, pseudo-marginal MH
stochastic Lotka–Volterra	2-d SDE	4	both, pointwise	variable	bridge, tempered SMC
Heston	2-d SDE	5	price only	variable	particle filter, tempered SMC
Merton jump-diffusion	jump SDE	5	full	variable	Poisson-mixture MCMC
FitzHugh–Nagumo	2-d ODE	4	v only	variable	exact-likelihood nested sampling
Hodgkin–Huxley	4-d ODE	4	V only	variable	exact-likelihood nested sampling
SEIR	4-d ODE	5	I, D	variable	exact-likelihood MCMC
pharmacokinetics (Bateman)	ODE (closed form)	3	concentration	variable	log-normal MCMC
Fisher–KPP	PDE (64-point grid)	2	3 sensors	variable	PDE-likelihood MCMC

Table 1: The benchmark suite. “obs.” names the observed coordinate where the state is only partly visible; every system is trained and evaluated in the design-amortized mode, so each observation set carries its own (time, sensor) points. The last column names the instrument that builds that system’s reference posterior; the choice follows the structure of the likelihood (Sec. 5.1).

reference standard deviation is ours, since the parameters carry different physical units. Distribution-free discrepancies and the classifier two-sample test of Lueckmann et al. (2021) rank the systems identically; the choice with consequences is the normalization, which the notation above makes explicit.

6 Benchmark suite

The suite contains fourteen systems (Table 1); each is defined below with its equations, parameter ranges, and observation model. Variable-design training, with observation subsets resampled at every optimizer step, is the package default and the recommended mode: beyond enabling inference at any design, the resampling acts as data augmentation over designs.

All SDEs are integrated by Euler–Maruyama on the stated fine grid; ODEs by RK4 (Hodgkin–Huxley by a semi-implicit scheme for the gating variables); the PDE by finite differences. Every system is trained and evaluated in the design-amortized mode of Sec. 2: the observation times (and, for the PDE, sensor indices) of every observation set are drawn at random, and one trained network serves all of them. The *classical estimator* listed for each system is the standard non-Bayesian method a practitioner would apply; all of them return point estimates, not distributions, and they serve as an accuracy floor for the posterior mean. Three recur and are defined here once. *Kramers–Moyal regression* estimates the drift and squared diffusion from the conditional first and second moments of the observed increments ($f(x) \approx \mathbb{E}[\Delta X \mid X = x]/\Delta t$, $\sigma^2(x) \approx \mathbb{E}[\Delta X^2 \mid X = x]/\Delta t$) and fits the parametric form to these estimates by least squares. *Multi-start nonlinear least squares* fits the deterministic solve of the system to the observed trajectory by minimizing squared error over parameters, restarted from many random initial points to escape local minima. *Exact maximum likelihood* is available where the transition density is known in closed form. Fixed-design observations are token sets over the union of the stated channels; “fast/slow” channels observe every step over a short window and every j -th step over the horizon respectively. Priors are uniform on the stated boxes.

Every system runs in the design-amortized mode: an observation set draws K random (time, channel) points on the stated simulation grid, with K between the per-system minimum and 128 under the design-size law of Sec. 4. The “channels” entries below additionally define the conventional fixed grids used by the calibration-screen gallery of Appendix A for the systems that appear there.

Linear-Gaussian. $y = Am + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.5^2 I_6)$, with a fixed correlated design matrix $A \in \mathbb{R}^{6 \times 4}$; $m \in [-3, 3]^4$. Exact Gaussian posterior; the classical estimator is the posterior mean itself. This system checks the flow engine in isolation from the encoder.

Geometric Brownian motion. $dS = \mu S dt + \sigma S dW$, $S_0 = 1$; $dt = 0.01$, 500 steps (five “years”); channels: fast (1×60), slow (10×40). Prior $\mu \in [-0.2, 0.4]$, $\sigma \in [0.1, 0.6]$. Classical: exact MLE on log-returns. Reference: conjugate posterior (Appendix C).

Ornstein–Uhlenbeck. $dX = -\theta X dt + \sigma dW$, stationary start $X_0 \sim \mathcal{N}(0, \sigma^2/2\theta)$; $dt = 0.02$, 500 steps; channels: fast (1×50), slow (20×24). Prior $\theta \in [0.3, 3]$, $\sigma \in [0.2, 1.5]$. Classical: exact MLE from per-gap Gaussian transitions.

Cox–Ingersoll–Ross. $dX = a(b - X) dt + \sigma\sqrt{X} dW$; $dt = 0.02$, 500 steps; channels as OU. Prior $a \in [0.3, 3]$, $b \in [0.3, 1.5]$, $\sigma \in [0.1, 0.5]$. Classical: Euler pseudo-likelihood MLE (Cox et al., 1985).

Double well. $dX = (\theta_1 X - \theta_2 X^3) dt + \sigma dW$; $dt = 0.01$, 1000 steps, noise strong enough for barrier crossings; channels: fast (1×80), slow (25×36). Prior $\theta_{1,2} \in [0.5, 3]$, $\sigma \in [0.4, 1.2]$. Classical: Kramers–Moyal regression (defined above).

Polynomial-drift SDE. $dX = (c_0 + c_1 X + c_2 X^2 + c_3 X^3) dt + \sigma dW$: drift identification on a dense polynomial basis, the setting of SINDy (Brunton et al., 2016) without its sparsity prior (a sparse-coefficient variant is discussed in Sec. 8); $dt = 0.01$, 1000 steps; channels: fast (1×80), slow (20×45). Prior $c_0, c_2 \in [-0.5, 0.5]$, $c_1 \in [-2, 0]$, $c_3 \in [-1, -0.1]$, $\sigma \in [0.3, 0.8]$. Classical: least squares of the Kramers–Moyal drift estimate on the polynomial basis, the dense-basis version of SINDy regression.

Stochastic Lotka–Volterra. Prey/predator pair with multiplicative noise, $dx_1 = (\alpha x_1 - \beta x_1 x_2) dt + 0.05 x_1 dW_1$, $dx_2 = (\delta x_1 x_2 - \gamma x_2) dt + 0.05 x_2 dW_2$; $dt = 0.01$, 600 steps. Each observation token is one (time, species) point, so a design of K points spreads over both species; $K \in [4, 128]$. Prior $\alpha, \gamma \in [0.8, 1.5]$, $\beta, \delta \in [0.4, 1.2]$. Classical: multi-start nonlinear least squares (defined above).

Heston. $dS = \mu S dt + \sqrt{v} S dW_1$, $dv = \kappa(\bar{v} - v) dt + \xi \sqrt{v} dW_2$, $\text{corr}(dW_1, dW_2) = \rho$ (Heston, 1993); price observed, volatility latent; $dt = 0.01$, 500 steps, designs of $K \in [2, 128]$ random times. Prior $\mu \in [-0.1, 0.3]$, $\kappa \in [0.5, 5]$, $\bar{v} \in [0.02, 0.3]$, $\xi \in [0.1, 1]$, $\rho \in [-0.9, 0]$.

Merton jump-diffusion. $dS/S = \mu dt + \sigma dW + d(\sum_i (e^{J_i} - 1))$ with Poisson jump rate λ and $J_i \sim \mathcal{N}(\mu_J, s_J^2)$ (Merton, 1976); $dt = 0.01$, 500 steps, $K \in [2, 128]$. Prior $\mu \in [-0.1, 0.3]$, $\sigma \in [0.1, 0.5]$, $\lambda \in [0.2, 3]$, $\mu_J \in [-0.3, 0.3]$, $s_J \in [0.05, 0.4]$. Reference: Poisson-mixture transition density (Appendix C).

FitzHugh–Nagumo. The classical two-dimensional reduction of the neuronal excitability equations (FitzHugh, 1961; Nagumo et al., 1962): $\dot{v} = v - v^3/3 - w + I$, $\dot{w} = \epsilon(v + a - bw)$; only v is observed, at arbitrary times drawn per observation set ($K \in [4, 64]$, horizon 40, $dt = 0.05$), with observation noise 0.05; the recovery variable w is never seen. Prior $a, b \in [0.5, 0.9]$, $\epsilon \in [0.05, 0.3]$, $I \in [0, 0.5]$. This is deliberately harder than the community’s toy configuration (e.g. the three-parameter, fully observed variant shipped with `pints`, Clerx et al. 2019): partial observation leaves a latent coordinate, the drive I is a fourth unknown, and the resulting posteriors are the needle-and-mode geometry documented in Sec. 5.1. Classical: multi-start nonlinear least squares (defined above).

Hodgkin–Huxley. Standard four-dimensional membrane model (Hodgkin and Huxley, 1952); voltage observed at random times; parameters $(g_{\text{Na}}, g_K, g_L, I)$ with prior $[60, 180] \times [10, 50] \times [0.05, 0.5] \times [4, 12]$; 3000 integration steps, $K \in [4, 96]$.

SEIR with mortality. Five-compartment stochastic epidemic (two-phase contact rate $\beta_1 \rightarrow \beta_2$, incubation α , recovery γ_r , mortality γ_d); infected and deceased compartments observed, each token one (time, compartment) point, $K \in [4, 64]$; 60-day horizon ($dt = 0.1$), observation noise 0.004. Prior $\beta_1 \in [0.2, 0.6]$, $\beta_2 \in [0.05, 0.3]$, $\alpha \in [0.2, 0.5]$, $\gamma_r \in [0.05, 0.2]$, $\gamma_d \in [0.005, 0.05]$. Classical: nonlinear least squares on (I, D) .

Pharmacokinetics. The standard one-compartment oral-dosing model of pharmacokinetics (e.g. Gabrielsson and Weiner, 2000): absorption at rate k_a , elimination at rate k_e , distribution volume V , giving the Bateman concentration curve, concentration $c(t) = \frac{D_0 k_a}{V(k_a - k_e)}(e^{-k_e t} - e^{-k_a t})$ with dose $D_0 = 500$; log-normal assay noise (10%); horizon 24 h, $K \in [3, 64]$ irregular draws. Prior $k_a \in [0.5, 4]$, $k_e \in [0.05, 0.4]$, $V \in [20, 100]$. Reference: log-normal likelihood MCMC.

Fisher–KPP. $\partial_t \theta = D \partial_{xx} \theta + r \theta(1 - \theta)$ on $[0, 1]$ with reflecting boundaries, 64-point finite-difference grid, Gaussian initial bump; three fixed sensors at $x = 16/63, 32/63, 48/63$ observed at random (time, sensor) pairs with noise 0.02; $dt = 0.01$, 200 steps, $K \in [4, 96]$. Prior $D \in [0.001, 0.01]$, $r \in [2, 10]$. Reference: Gaussian observation model around the deterministic solve; one likelihood evaluation is one PDE solve.

7 Experimental results

This section is organized as an experimental study: the protocol (references, baselines, model sizes, budgets) is fixed first; the main results (posterior overlays and the global metric on every system) follow; then the evaluation of design amortization and the cost of training and inference.

7.1 Experimental protocol

References and baselines. Every accuracy number in this section is the FID of Sec. 5.1 against a reference posterior computed on the same observed points, reported as the median over the system’s evaluation set: a frozen collection of observation-set instances (200 where the reference is closed form, 100 where it is cheap MCMC, 32 where each reference costs minutes of validated MCMC), built once with two-chain validation (Sec. 5.1) and reused for every number in this report, so any two entries differ only in the model being scored. The reference engines span more than six orders of magnitude in per-instance cost, from the closed-form GBM posterior (<1 ms) through exact-likelihood MCMC (seconds) to nested sampling and pseudo-marginal chains (minutes, and an hour on the hardest instances); the full accounting, alongside the amortized model’s own training and inference cost, is Table 3 in Sec. 7.4. We do not report accuracy comparisons with other amortized packages: their estimators consume fixed-length summary vectors, at package defaults we could not obtain converged posteriors from them on raw irregular time series, and building fair problem-specific summary networks for every system is outside the scope of this report; the validated reference posteriors, not another package, are the accuracy record throughout. Simulation-based calibration serves as the screen for the fixed-design gallery systems that carry no validated reference; its per-parameter results are in Appendix A.

Models and training. For each reference system we train the same architecture at three sizes (tiny, 0.1M parameters; the default small, 0.5M; big, 2.3M) plus two miniature sizes (nano, 26k; pico, 7k) used to probe the low-capacity end. All runs follow the procedure of Sec. 4. Simulation budgets range from 5,000 to 480,000 per system. The size sweep under the default recipe opens the results (Table 2). Observation designs are random at training and at evaluation wherever the system defines a design distribution; fixed designs appear only where a reference requires them.

7.2 Main comparison: quality, model size, data, and baselines

One comparison, run identically on every task, opens the results: the package’s default recipe (Muon with the batch step rule, batch 512, a deliberately generous 90,000 steps with validated stopping; Sec. 4), trained at five model sizes on every system with a validated random-design reference, three seeds at the top three sizes, and scored on the frozen evaluation sets. Every system of the suite has a row, and every row carries a validated reference built by the engine matched to its likelihood (Sec. 5.1): closed forms where they exist, adaptive Metropolis or nested sampling where the likelihood is exact, and pseudo-marginal MH or tempered SMC over unbiased estimates where the transition density has no closed form. Hodgkin–Huxley was built twice, once by tempered SMC and once by nested sampling, and the two size sweeps agree to less than the seed spread at every model size, the strongest cross-check in this report. Table 2 is the result; Fig. 6 shows the corresponding posteriors.

On GBM every size from tiny up sits within two floors of the evaluation set’s own resolution (0.0028–0.0036 against 0.0018), and linear-Gaussian, double well, and SEIR land within a few floors at the larger sizes, so those cells bound the error from above rather than measure it. Ornstein–Uhlenbeck (~ 0.028) and Cox–Ingersoll–Ross (0.07–0.08) are independent of model size, so their residuals belong to the task rather than to capacity or recipe. FitzHugh–Nagumo and Hodgkin–Huxley improve most strongly with model size ($0.45 \rightarrow 0.036$ from pico to small, $9.8 \rightarrow 0.31$ across the five sizes with the largest model still improving) and carry the largest residuals of the suite; the stochastic Lotka–Volterra residual (0.80) is concentrated almost entirely in one coordinate (Fig. 5) and is not explained by capacity (0.95 at the big size). Validated stopping has a measurable effect: under the margin rule of Sec. 4, the best-on-validation

system	floor	FID _σ at model size (parameters)				
		pico (7k)	nano (26k)	tiny (98k)	small (0.5M)	big (2.3M)
linear-Gaussian	0.0030	0.0660 ± 0.0131	0.0148 ± 0.0003	0.0068 ± 0.0009	0.0045 ± 0.0001	0.0042 ± 0.0005
geometric Brownian motion	0.0018	0.0064	0.0039	0.0036 ± 0.0004	0.0028 ± 0.0001	0.0035 ± 0.0006
Ornstein–Uhlenbeck	0.0013	0.0297	0.0281	0.0294 ± 0.0026	0.0282 ± 0.0031	0.0281 ± 0.0017
Cox–Ingersoll–Ross	0.0028	0.0786 ± 0.0178	0.0823 ± 0.0131	0.0733 ± 0.0110	0.0809 ± 0.0071	0.0700 ± 0.0036
pharmacokinetics	0.0023	0.0846	0.0319	0.0340 ± 0.0021	0.0262 ± 0.0062	0.0251 ± 0.0024
Merton jump-diffusion	0.0065	0.0663	0.0299	0.0219 ± 0.0007	0.0247 ± 0.0016	0.0267 ± 0.0050
Fisher–KPP	0.0100	0.0705 ± 0.0161	0.0415 ± 0.0033	0.0377 ± 0.0054	0.0307 ± 0.0036	0.0282 ± 0.0035
FitzHugh–Nagumo	0.0052	0.4495 ± 0.0596	0.2367 ± 0.1194	0.0864 ± 0.0394	0.0357 ± 0.0030	0.0406 ± 0.0265
SEIR	0.0069	0.1511 ± 0.0354	0.0642 ± 0.0112	0.0185 ± 0.0024	0.0114 ± 0.0027	0.0097 ± 0.0015
double well	0.0028	0.0201 ± 0.0047	0.0105 ± 0.0000	0.0089 ± 0.0009	0.0093 ± 0.0009	0.0139 ± 0.0020
polynomial-drift SDE	0.0055	0.0342 ± 0.0043	0.0200 ± 0.0045	0.0205 ± 0.0034	0.0271 ± 0.0041	0.0386 ± 0.0113
Heston	0.0052	0.1829 ± 0.0537	0.0753 ± 0.0107	0.0457 ± 0.0007	0.0508 ± 0.0044	0.0543 ± 0.0053
Hodgkin–Huxley	0.0049	9.76 ± 1.41	5.83 ± 0.87	2.37 ± 0.32	1.52 ± 0.46	0.313 ± 0.034
stochastic Lotka–Volterra	0.0073	7.29 ± 0.23	3.23 ± 0.58	1.42 ± 0.20	0.80 ± 0.10	0.95 ± 0.23

Table 2: The main result: every system of the suite against its validated reference. Entries are FID_σ (median over the frozen evaluation set of Sec. 7.1; $K = 20$; Merton $K = 56$; Fisher–KPP $K = 40$ with 4,000-draw PDE-likelihood chains, hence its higher floor; linear-Gaussian references are exact, no MCMC), one design-amortized network per cell trained with the package default recipe at 120,000 simulations; $\pm$ is the half-range over independent training seeds (single-seed cells at pico and nano carry none); the best cell of each row is set in bold.

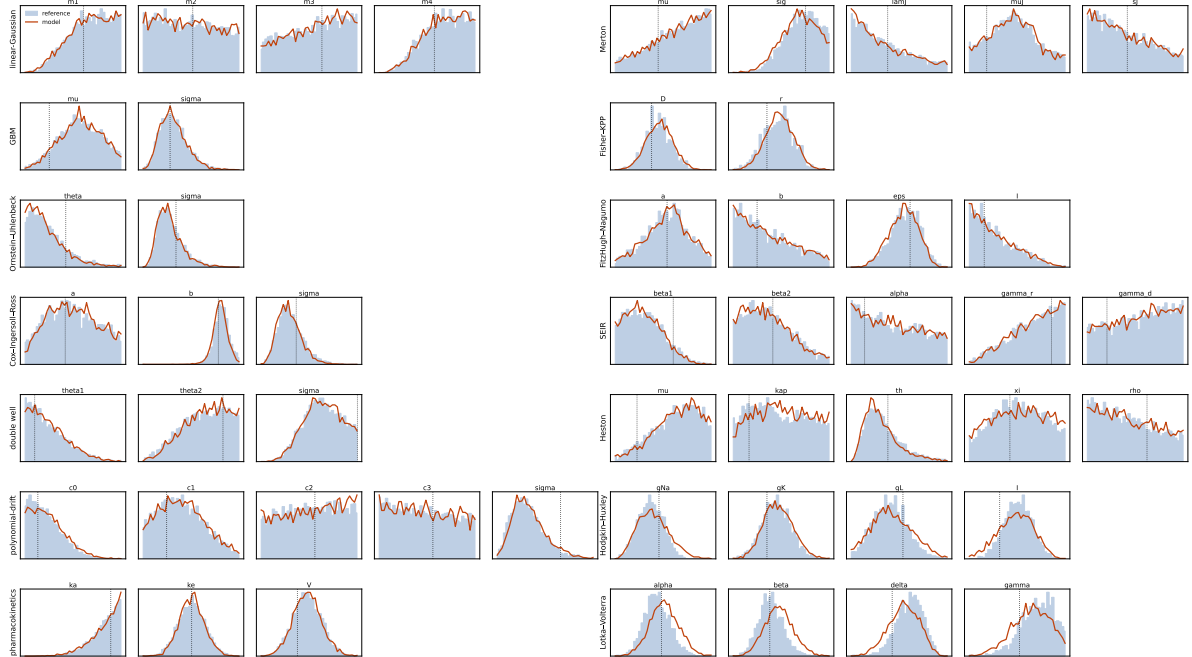


Figure 5: One-dimensional posterior marginals, model (line) against reference (filled), for every system of Table 2: the median-FID_σ test instance of each, 4,000 draws per side, dotted line at the generating parameter; each panel’s axis spans its sample range, not the prior box (Fig. 6 shows the prior-relative scale). Rows are systems, columns their parameters. This is the same comparison the table reduces to one number per cell, and it shows what that number cannot: where the model is exact (linear-Gaussian, GBM, double well), where it is faithful but slightly over-dispersed, and where a single coordinate carries the whole error (Lotka–Volterra’s β , Hodgkin–Huxley’s conductances on sharp instances).

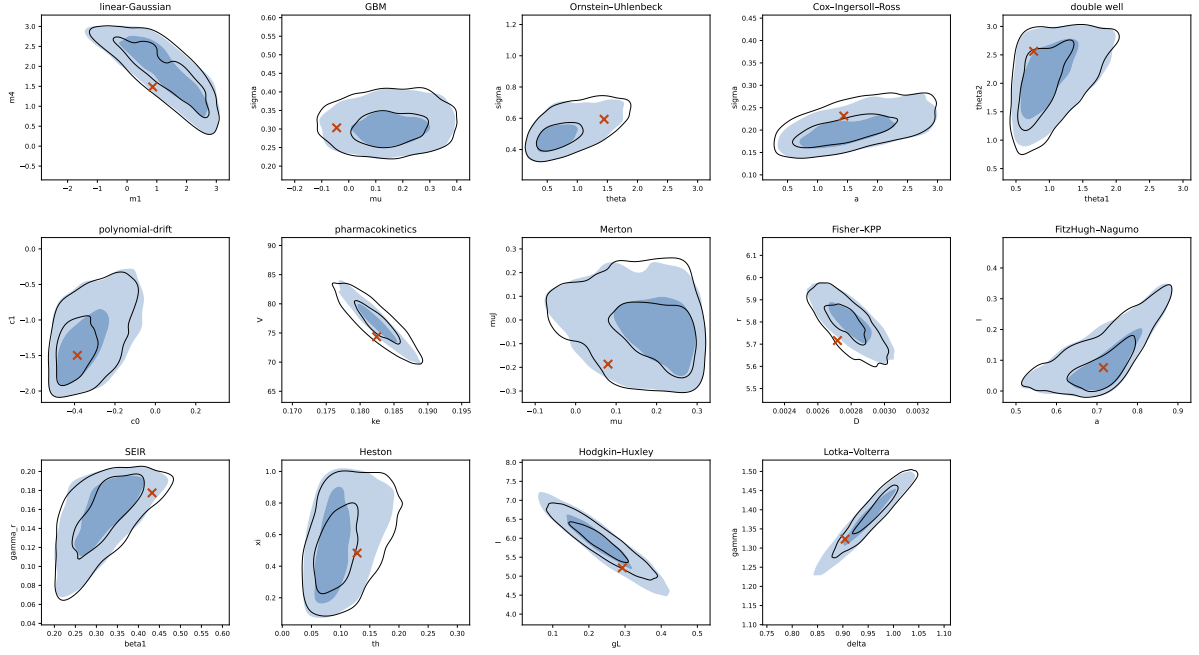


Figure 6: Joint posteriors of `amortix` (filled contours: 39% and 86% highest-density regions) against exact or validated reference posteriors (open contours, same levels), for the median- FID_σ test instance of every system of Table 2; the big model of the default recipe, 4,000 draws per side; the cross marks the true parameters. Each panel shows that system’s most correlated parameter pair, so what is being compared is the shape of the dependence and not only its marginal spread. Axes span the sample range of each panel, not the prior box: the prior is $4\text{--}107\times$ wider than the posterior standard deviation across the panels shown, and the sharp FitzHugh–Nagumo and Hodgkin–Huxley instances of Sec. 5.1 reach $\sim 230\times$, so the plotted shapes carry no information about absolute sharpness.

checkpoint replaced the final weights in half of the 194 training runs behind Table 2, in 13 of 15 on Cox–Ingersoll–Ross and in 15 of 15 on FitzHugh–Nagumo (Appendix B).

Figure 6 shows the object the package produces, the joint posterior itself, for the median-error test instance of every reference-block system, model against reference, under the same recipe and checkpoints as Table 2.

7.3 Amortization over observation designs

The package’s central claim is that one trained network answers $p(m | S)$ for *every* observation design S , at any number and placement of points. Fig. 7 tests the claim on the six systems of the suite where a per-design reference can be computed and validated: each test observation set draws a fresh random design, the reference posterior is recomputed on exactly those points, and FID is evaluated per design-size bucket. In our experiments, amortization over designs is not a trade-off against accuracy: wherever a network trained on a single fixed design exists for comparison (the stars in Fig. 7), the amortized network is comparable to it at that design size and is the only one of the two that answers anywhere else on the K axis. It also removes a failure mode, since resampling the observation subset at every optimizer step doubles as data augmentation. The two systems with elevated FID are not noise; the next paragraph analyzes them.

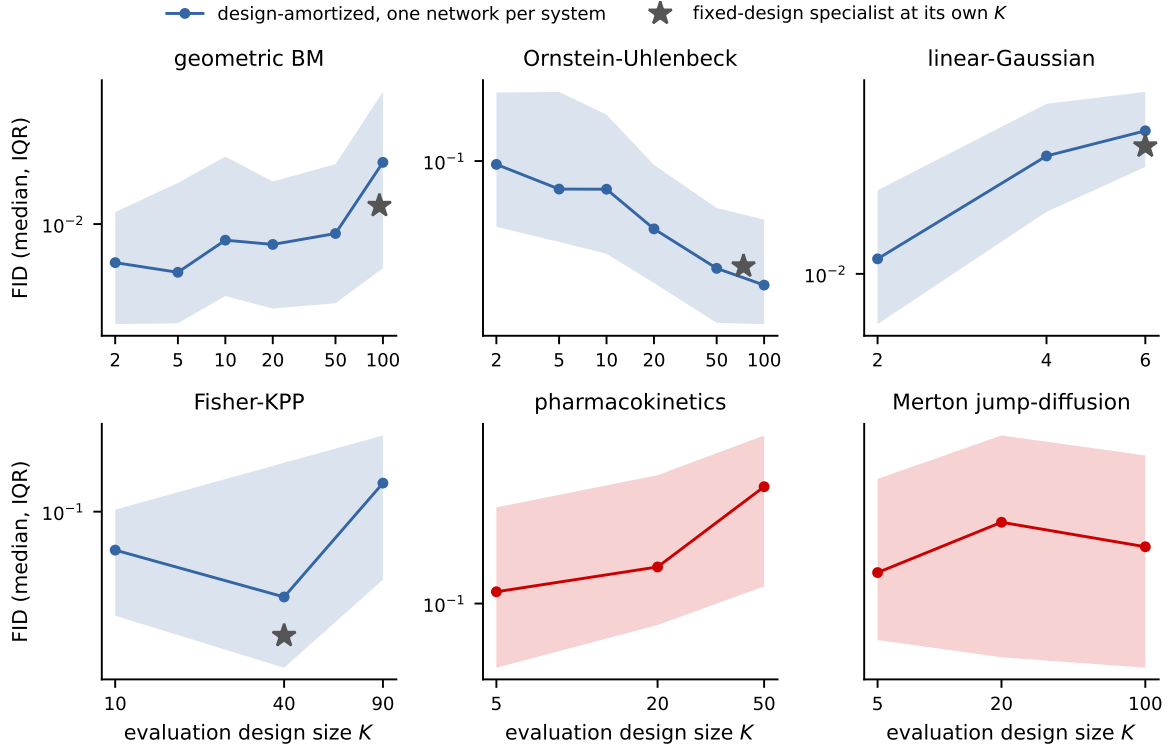


Figure 7: FID as a function of the evaluation design size, for one design-amortized network per system: median (line) and interquartile range (band) over the test observation sets of each bucket (100 per bucket for GBM, OU, and linear-Gaussian, 32 for the MCMC-referenced systems), each set with a fresh random design and a per-design reference recomputed on exactly those points. These are the six systems whose references were built on a grid of design sizes; the rest of the suite has references at one design size (Table 2), which is enough to score a model but not to trace a curve. Stars: networks of the same architecture trained on a single fixed design, which exist only at their own K ; shown for context. The Merton and pharmacokinetics curves use the converged references of Sec. 5.1; the pharmacokinetic rise with K is the precision floor analyzed in the text. Both axes logarithmic; note the different vertical scales.

Why FID_{σ} rises with the design size. FID_{σ} is a ratio, and on pharmacokinetics ($0.07 \rightarrow 0.34$ from $K = 5$ to 50, Fig. 7) only its denominator moves. Measured on the same evaluation sets, in prior-range units: the network’s absolute posterior-mean error is flat in K ($\varepsilon_{\Delta} = 1\text{--}12 \times 10^{-3}$, no trend), the reference posterior contracts by the $\sqrt{K}$ of a regular model (widths fall $2.8\text{--}5.1\times$, against $\sqrt{10} \approx 3.2$), and the

engine	systems	one posterior
closed form / conjugate	linear-Gaussian, GBM	<1 ms
Poisson-mixture / log-normal MCMC	Merton, pharmacokinetics	5–7 s
exact-likelihood MCMC	OU, CIR	6–34 s
tempered SMC, particle-filter likelihood	Heston	58 s
nested sampling (dynesty)	FitzHugh–Nagumo	≈2 min
PDE-likelihood MCMC	Fisher–KPP	186 s
tempered SMC, bridge likelihood	stochastic Lotka–Volterra	≈4 min
exact-likelihood MCMC	SEIR	≈16 min
nested sampling (dynesty)	Hodgkin–Huxley	≈18 min
pseudo-marginal MH, bridge likelihood	double well, polynomial-drift	≈38 min
amortix small (GPU), after training	any of the above	0.07 s
amortix small (laptop CPU)		0.43 s

Table 3: Per-instance cost of one reference posterior at the settings that built the frozen evaluation sets: MCMC rows are single chains of $1.5\text{--}2 \times 10^5$ draws (Fisher–KPP 4,000; double well and polynomial drift 60,000 pseudo-marginal draws), thinned to the 4,000 stored draws; the two-chain admission of Sec. 5.1 runs each engine two to three times. Every engine runs on the CPU (one to a few threads on one server core) except the Heston SMC filter, which runs on the GPU at the production population (16,384 θ -particles over a 4,096-particle filter; median over 96 runs, spread 28–144 s). The CPU chains are single-instance timings; the dynesty rows are medians over the 32 test instances, reconstructed from the per-job wall times of the reference builds (**nlive**=1000), with a spread across instances of about four on FitzHugh–Nagumo and about twenty on Hodgkin–Huxley, whose hardest instances take an hour. Training the amortized network costs 58–82 minutes on one H200 GPU (text above); after training, one posterior costs the entry in the last two rows.

bias term $\sum_i (\varepsilon_{\Delta,i}/w_{\Delta,i})^2$ grows accordingly, $0.025 \rightarrow 0.121$; in the prior normalization the same model reads $\text{FID}_{\Delta} \approx 3 \times 10^{-4}$ at the densest designs. The posterior widths themselves stay within 1–8% of the reference at every K : what grows is the visibility of the network’s fixed localization floor, $10^{-3}\text{--}10^{-2}$ of the prior range, once the data determine the parameters more precisely than that. The pharmacokinetic entries are therefore reported per design size rather than pooled: the narrowest-posterior instances read several times worse than the rest, and a median over one 32-instance evaluation set moves between 0.04 and 0.15 between otherwise identical training runs. The stochastic systems stay an order of magnitude above the floor even at $K = 100$; Fisher–KPP approaches it only at its densest bucket (0.126 at $K = 90$).

7.4 Cost of training and inference

Training under the default recipe (90,000 steps at batch 512, 120,000 simulations, validated stopping) costs 58–82 minutes per system at the default size on one H200 GPU, and 67–111 minutes at the big size (136 for Hodgkin–Huxley, whose simulator is the most expensive). The budget is deliberately generous: with the checkpoint chosen by validation, overshooting it costs time, never quality (Sec. 4), and at a 20,000-simulation budget training takes 4–8 minutes. The same run on a laptop CPU takes roughly four times longer and produces a bit-identical posterior, because all stochastic generation is kept on a CPU random number generator. At inference, 2,000 posterior samples for one observation set cost 72 ms batched (135 ms single) on the GPU and 426 ms on the laptop CPU, at any design size and placement.

Table 3 sets these numbers against the per-instance cost of the reference engines that built the evaluation sets, at the settings that built them: chains long enough to clear the two-chain floors of Sec. 5.1, i.e. reference grade rather than user grade (a 2,000-draw exact-likelihood chain costs 42–124 ms). The comparison cuts both ways. Where the likelihood costs microseconds, exact-likelihood MCMC produces a user-grade posterior in tens of milliseconds and a reference-grade one in seconds, cheaper than training the amortized network for any realistic number of observation sets, which is why those engines, not the network, are the accuracy record of this report. The relationship inverts where one likelihood evaluation is itself a simulation: a Fisher–KPP reference costs 186 s (a PDE solve per evaluation), a SEIR reference 16 minutes (an epidemic integration per evaluation), a Hodgkin–Huxley reference a median 18 minutes of nested sampling, and a double-well or polynomial-drift reference 38 minutes of pseudo-marginal MH. At those prices the entire training run amortizes within a handful of posteriors (two on the double well, four on SEIR and on Hodgkin–Huxley, about twenty on Fisher–KPP), and every posterior after that costs 70 ms. Sequential refinement and design search require posteriors by the hundred; that is the intended regime of the package.

system	per-parameter SBC p -values					
linear-Gaussian	m_1 : 0.06	m_2 : 0.60	m_3 : 0.38	m_4 : 0.56		
Ornstein-Uhlenbeck	θ : 0.28	σ : 0.77				
SEIR	β_1 : 0.33	β_2 : 0.22	α : 0.61	γ_r : 0.34	γ_d : 0.53	
geometric Brownian motion	μ : 0.26	σ : 0.44				
Cox-Ingersoll-Ross	a : 0.65	b : 0.001	σ : 0.73			
double well	θ_1 : 0.78	θ_2 : 0.54	σ : 0.88			
stochastic Lotka-Volterra	α : 0.07	β : 0.69	δ : 0.08	γ : 0.35		
FitzHugh-Nagumo	a : 0.19	b : 0.15	ϵ : 0.90	I : 0.63		
polynomial-drift SDE	c_0 : 0.06	c_1 : 0.96	c_2 : 0.46	c_3 : 0.80	σ : 0.50	

Table 4: Per-parameter SBC uniformity p -values for the standard-budget fixed-design gallery run (500 observation sets $\times$ 200 draws; $p > 0.05$ passes). The single failure is marked in bold.

8 Discussion and limitations

The evidence assembled here supports a specific claim: for dynamical-systems inverse problems, the input representation of an amortized posterior estimator is not a preprocessing detail but the dominant accuracy factor, and it can be made part of the model rather than part of the problem specification. With prescribed coordinates our own estimator acquires the failures documented in Sec. 3.3; the learnable representation removes them with one configuration for all fourteen systems, and the reference posteriors of Sec. 5 certify the result.

Limitations. Bayesian experimental design, the intended consumer of design amortization, is not demonstrated yet; the training procedure of Sec. 4 was built so that the required object $p(m | S)$ is already coherent across subsets S . No real-data study with posterior predictive checks is included. Priors are limited to factorized distributions with tractable marginal CDFs, the class the probit map (2) covers; correlated priors would require a different normalization, and priors with atoms at zero, which would admit a sparse-coefficient variant of the polynomial-drift system, are a planned extension. Finally, the two hardest systems are not yet at reference quality: Hodgkin-Huxley is still improving at the largest model size, and the Lotka-Volterra residual is not explained by capacity (Sec. 7.2), the clearest open problem in the suite.

A Controls and calibration screen

Controls. Two model-free controls accompany every result. The *prior-mean predictor* ignores the data entirely (exactly 25% mean absolute error under a uniform prior). The *ridge control* is a point predictor of the parameters: a degree-2 ridge regression on generic summary statistics of the token set: per token feature, the mean, standard deviation, mean square, mean absolute value, and the 0.1/0.5/0.9 quantiles, computed over however many observations are present. It recovers no distribution and is not meant to; its role is to detect a specific silent failure that the calibration screen cannot see: a posterior that ignores the data and returns the prior passes SBC perfectly, and only a point-accuracy control unmasks it.

Calibration of the fixed-grid gallery. Per-parameter SBC p -values for the standard-budget gallery run (40,000 simulations) are listed in Table 4. The protocol: for each system, 500 synthetic observation sets are simulated with known parameters; for each observation set the model produces 200 posterior draws, and the rank of the true parameter among the draws is recorded. A calibrated posterior yields uniform ranks, and each p -value tests this uniformity for one parameter. All parameters pass except Cox-Ingersoll-Ross b , whose deviation corresponds to an ≈ 0.15 sd location shift of the posterior in b , the largest calibration deviation in the gallery.

B Optimizer and stopping-rule measurements

Setup. Every number in this appendix comes from one configuration, so that only the optimizer variables change: the design-amortized GBM system, the big model (2.3M parameters), and scoring by FID_σ against the exact conjugate reference on a fixed six-bucket evaluation set (design sizes $K = 2, 5, 10, 20, 50, 95$; 32 observation sets per bucket; two-chain floors 0.0012–0.0019). Reported values are means over the six

buckets. Each training run occupies one H200 GPU; several comparisons hold the example budget (batch $\times$ steps) fixed on purpose, and state it.

Step size at the reference batch. At batch 512 and 9,000 steps (4.6M examples):

Muon step η	3×10^{-4}	10^{-3}	3×10^{-3}	10^{-2}	2×10^{-2}	6×10^{-2}
FID $_{\sigma}$	0.0133	0.0070	0.0065	0.0061	0.0085	diverges

The basin is broad and shallow between 3×10^{-3} and 10^{-2} , and the edge beyond it is hard. The default sits at 10^{-2} .

Scaling with batch size. Repeating the sweep at batches 64, 2,048, and 8,192 (example budget fixed within each comparison) locates the optimum at 3.5×10^{-3} , 10^{-2} (flat to 2×10^{-2}), and 2×10^{-2} respectively. From batch 64 to 512 the optimum moves by $\times 2.82$ against $\times 2.83$ predicted by $\sqrt{B}$; the fourth-root alternative is excluded rather than disfavored, since its prediction costs 0.0079 against 0.0066 at batch 64. Past batch $\sim 2,048$ the optimum stops moving: at 8,192 the square root would extrapolate to 4×10^{-2} , which scores worse than the capped 2×10^{-2} (0.0060 against 0.0052), and twice the extrapolation diverges. Hence the default rule of Sec. 4: $\eta(B) = \min(10^{-2} \sqrt{B/512}, 2 \times 10^{-2})$, square root below the critical batch, constant above it.

Transfer across width. At batch 512 and fixed budget, the step 10^{-2} against the hotter 2×10^{-2} across three widths: tiny 0.0089 vs 0.0087, small 0.0066 vs 0.0076, big 0.0061 vs 0.0085. At the hotter step accuracy is non-monotone in model size (big worse than small); at the step given by the rule it is ordered by capacity. “Wider is worse” can therefore be a statement about one number inside the optimizer rather than about capacity.

Steps against examples. At a fixed budget of 9.2M examples, batch $512 \times 18,000$ steps reads 0.0051, batch $2,048 \times 4,500$ reads 0.0045, and batch $8,192 \times 1,125$ collapses to 0.0076; at the same batch and step size, doubling the steps to 2,020 recovers 0.0060. Above a step floor the example budget governs quality; below it no example budget helps. The floor is per-system and spans an order of magnitude: the pharmacokinetic validation curve reaches its optimum near 66,000 steps at batch 512, where GBM’s shows no interior optimum up to 36,000; this fact motivates validated stopping in place of per-system budgets.

Validated stopping. The monitor’s resolution must be evaluated at the monitor’s own sample count: the GBM validation set resolves 0.0013 at 4,000 draws but only 0.0044 at 1,000. Both failure modes of the naive keep-the-best rule were measured before the margin was adopted. A monitor coarser than its model ships the deepest noise dip: 0.0053 on test against 0.0040 for the final weights. A resolving monitor still ships 0.0052, because under a cosine schedule mid-run checkpoints are systematically inflated, and an argmin over the whole curve mistakes that for a reversal. The margin rule (accept the best checkpoint only when it improves on the final weights by more than the monitor’s resolution) passes both cases, and a genuine late reversal clears it by an order of magnitude: on pharmacokinetics the validation curve turns from 0.1066 at step 66,222 to 0.1321 at 72,072, twenty floors, and the selected checkpoint recovers the difference.

C Reference posteriors

GBM conjugate posterior. With observations at indices $i_0 = 0 < i_1 < \dots$ of the fine grid, log-returns $r_k = \log(S_{i_{k+1}}/S_{i_k})$ over gaps τ_k are independent $\mathcal{N}((\mu - \sigma^2/2)\tau_k, \sigma^2\tau_k)$. In the coordinates $(b, \sigma^2) = (\mu - \sigma^2/2, \sigma^2)$ the model is a Gaussian location-scale family with sufficient statistics $T = \sum_k \tau_k$, $R_1 = \sum_k r_k$, and $SS_w = \sum_k r_k^2/\tau_k - R_1^2/T$; draws from the standard conjugate posterior ($\sigma^2 = SS_w/\chi_{n-1}^2$, $b | \sigma^2 \sim \mathcal{N}(R_1/T, \sigma^2/T)$) are transformed to (μ, σ) and importance-resampled to the uniform box prior with the appropriate Jacobian weight. The sampler was validated against exact-likelihood MCMC on the same designs: over 24 comparisons, worst posterior-mean discrepancy 0.099 sd, width ratios 0.91–1.03.

Adaptive Metropolis. Random-walk Metropolis on the box, proposal $x' = x + sC\xi$, $\xi \sim \mathcal{N}(0, I)$. During burn-in (default $\max(500, n/2)$ iterations) the scalar s is adapted toward a 0.25 acceptance rate in blocks of 50 with a decaying Robbins–Monro gain, and once sufficient burn-in states are collected the shape C is replaced by the Cholesky factor of the empirical covariance rescaled by $2.38/\sqrt{d}$ (Haario et al., 2001); adaptation is frozen for the retained phase. Out-of-box proposals are rejected, which samples the box-truncated target exactly since the proposal is symmetric. Likelihoods: per-gap Euler–Maruyama Gaussian transitions for OU (matching the generative process, plus the stationary density of X_0); the exact GBM log-return density; the Poisson-mixture transition density for Merton (truncated at the numerically negligible jump count); log-normal residuals around the Bateman curve; a Gaussian observation model around the deterministic PDE solve for Fisher–KPP. Every chain used as a reference is paired with a second chain from a well-separated start; between-chain posterior-mean discrepancies are reported alongside the results they certify.

Nested sampling, SMC, and particle-filter likelihoods. FitzHugh–Nagumo and Hodgkin–Huxley references are drawn with `dynesty` (static nested sampling; the prior transform is the uniform box; `nlive`=1000; the admission partner runs `nlive`=2000 from an independent seed, and the pair must agree below the two-chain threshold of Sec. 5.1). Stochastic Lotka–Volterra and Heston use adaptive tempered SMC (Del Moral et al., 2006): the inverse-temperature step is chosen at each stage so the effective sample size stays at half the population, followed by systematic resampling and random-walk rejuvenation at the current temperature; admission is by population doubling (N against $2N$ from independent seeds). Their likelihoods are unbiased Monte-Carlo estimates (a Durham–Gallant bridge sampler for Lotka–Volterra (Durham and Gallant, 2002), a bootstrap particle filter running the simulator’s own clamped-Euler update for Heston; Sec. 5.1), at particle counts chosen so the estimator’s spread at fixed parameters is a fraction of a nat. Double well and polynomial drift use the same bridge estimator inside the pseudo-marginal variant (Andrieu et al., 2010) of the adaptive Metropolis sampler above.

D Correspondence with package options

in this report	in the package
point tokens	<code>embed="wpoint"</code>
pair tokens + set conditioning	<code>embed="wfilm"</code>
warped values/increments (fixed designs)	<code>embed="wbasis" / "wdiff"</code>
attention phases from physical time	<code>rope="time"</code>
fresh designs each optimizer step	<code>fit(retokenize=...) via make_retokenizer()</code>
Muon with the batch step law	<code>fit(optimizer="muon"); muon_lr=None</code> → the measured law
validated stopping with margin	<code>fit(keep_best=True, monitor_floor=...)</code>
evaluation-set resolution at n draws	<code>evaluation.floor_at</code>
variable-design problem wrapper	<code>amortix.designs.DesignProblem</code>
benchmark systems	<code>amortix.problems (DESIGN_Z00)</code>
calibration screen	<code>sbc_design</code>
reference machinery	<code>amortix.mcmc, .smc, .bridgelik, .hestonlik</code> + likelihood factories

All defaults named in this report are selected automatically by `FlowPosterior(problem)`; the non-default options are retained as reproducible controls.

References

- C. Andrieu, A. Doucet, R. Holenstein. Particle Markov chain Monte Carlo methods. *JRSS B* 72(3), 2010.
- J. Boelts, M. Deistler, M. Gloeckler, et al. sbi reloaded: a toolkit for simulation-based inference workflows. *arXiv:2411.17337*, 2024.
- S. L. Brunton, J. L. Proctor, J. N. Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *PNAS* 113(15), 2016.
- M. Clerx, M. Robinson, B. Lambert, C. L. Lei, S. Ghosh, G. R. Mirams, D. J. Gavaghan. Probabilistic inference on noisy time series (PINTS). *Journal of Open Research Software* 7(1), 2019.
- J. C. Cox, J. E. Ingersoll, S. A. Ross. A theory of the term structure of interest rates. *Econometrica* 53(2), 1985.
- K. Cranmer, J. Brehmer, G. Louppe. The frontier of simulation-based inference. *PNAS* 117(48), 2020.
- T. Darcet, M. Oquab, J. Mairal, P. Bojanowski. Vision transformers need registers. *ICLR*, 2024.
- P. Del Moral, A. Doucet, A. Jasra. Sequential Monte Carlo samplers. *JRSS B* 68(3), 2006.
- G. B. Durham, A. R. Gallant. Numerical techniques for maximum likelihood estimation of continuous-time diffusion processes. *Journal of Business & Economic Statistics*, 20(3):297–338, 2002.
- R. FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal* 1(6), 1961.
- J. Gabrielsson, D. Weiner. *Pharmacokinetic and Pharmacodynamic Data Analysis: Concepts and Applications*. Swedish Pharmaceutical Press, 2000.
- M. Gloeckler, M. Deistler, C. Weibach, F. Wood, J. H. Macke. All-in-one simulation-based inference. *ICML*, 2024.
- D. Greenberg, M. Nonnenmacher, J. Macke. Automatic posterior transformation for likelihood-free inference. *ICML*, 2019.
- H. Haario, E. Saksman, J. Tamminen. An adaptive Metropolis algorithm. *Bernoulli* 7(2), 2001.
- J. Hermans, A. Delaunoy, F. Rozet, A. Wehenkel, V. Begy, G. Louppe. A trust crisis in simulation-based inference? Your posterior approximations can be unfaithful. *TMLR*, 2022.
- S. L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *Rev. Financial Studies* 6(2), 1993.
- A. L. Hodgkin, A. F. Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *J. Physiology* 117(4), 1952.
- M. Horn, M. Moor, C. Bock, B. Rieck, K. Borgwardt. Set functions for time series. *ICML*, 2020.
- K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, J. Bernstein. Muon: an optimizer for hidden layers in neural networks, 2024. kellerjordan.github.io/posts/muon.
- Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. Flow matching for generative modeling. *ICLR*, 2023.
- J.-M. Lueckmann, J. Boelts, D. Greenberg, P. Gonçalves, J. Macke. Benchmarking simulation-based inference. *AISTATS*, 2021.
- R. C. Merton. Option pricing when underlying stock returns are discontinuous. *J. Financial Economics* 3(1–2), 1976.
- J. Nagumo, S. Arimoto, S. Yoshizawa. An active pulse transmission line simulating nerve axon. *Proceedings of the IRE* 50(10), 1962.
- G. Papamakarios, I. Murray. Fast ϵ -free inference of simulation models with Bayesian conditional density estimation. *NeurIPS*, 2016.
- W. Peebles, S. Xie. Scalable diffusion models with transformers. *ICCV*, 2023.
- E. Perez, F. Strub, H. de Vries, V. Dumoulin, A. Courville. FiLM: Visual reasoning with a general conditioning layer. *AAAI*, 2018.
- S. T. Radev, U. K. Mertens, A. Voss, L. Ardizzone, U. Köthe. BayesFlow: Learning complex stochastic models with invertible neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* 33(4), 2020.
- S. N. Shukla, B. M. Marlin. Multi-time attention networks for irregularly sampled time series. *ICLR*, 2021.
- J. Skilling. Nested sampling for general Bayesian computation. *Bayesian Analysis* 1(4), 2006.
- J. S. Speagle. dynesty: a dynamic nested sampling package for estimating Bayesian posteriors and evidences. *MNRAS* 493(3), 2020.
- J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568, 2024.
- S. Talts, M. Betancourt, D. Simpson, A. Vehtari, A. Gelman. Validating Bayesian inference algorithms with simulation-based calibration. *arXiv:1804.06788*, 2018.
- J. Wildberger, M. Dax, S. Buchholz, S. R. Green, J. H. Macke, B. Schölkopf. Flow matching for scalable simulation-based inference. *NeurIPS*, 2023.
- I.-K. Yeo, R. A. Johnson. A new family of power transformations to improve normality or symmetry. *Biometrika* 87(4), 2000.